

rapport BDA Phase 1 exploration et SQL avancé

Rapport de Projet BDA - Phase 1 : Exploration et SQL Avancé

Projet : Plateforme Web de Découverte de Films (CinéExplorer) **Module :** 4A-BDA – Bases de Données Avancées Armen ASATRYAN

1. Introduction

Dans le cadre du développement de la plateforme *CinéExplorer*, la première phase du projet consiste à maîtriser les données brutes fournies par IMDB. L'objectif est double : comprendre la structure et la qualité des données à travers une analyse exploratoire, puis concevoir une base de données relationnelle (SQLite) performante capable de répondre à des requêtes analytiques complexes. Ce rapport détaille la méthodologie, de l'ingestion des CSV à l'optimisation par indexation.

2. Exploration des Données (T1.0)

L'analyse des fichiers CSV bruts (dataset `imdb-small`) a permis de mettre en évidence la structure du jeu de données et d'identifier les traitements nécessaires avant l'importation.

2.1 Observations Clés sur la Qualité des Données

L'utilisation de la bibliothèque *Pandas* nous a permis de relever les points suivants :

- **Formatage des Colonnes** : Une particularité technique a été identifiée dans les en-têtes des fichiers CSV, où les noms de colonnes apparaissaient sous forme de tuples (ex: `('mid',)`). Un script de nettoyage a été développé pour normaliser ces noms.
- **Valeurs Manquantes** : La colonne `endYear` (table `Movies`) est vide à 100%, et `deathYear` (table `Persons`) est manquant pour environ 85% des entrées, ce qui est cohérent (beaucoup de personnes sont encore en vie).
- **Intégrité Référentielle** : L'analyse des clés étrangères a révélé un faible pourcentage d'orphelins (ex: des réalisateurs pointant vers des films absents du dataset "small"). Ces cas ont été traités lors de l'import.

2.2 Visualisations

Voici la distribution des données qui orientera nos choix de modélisation :

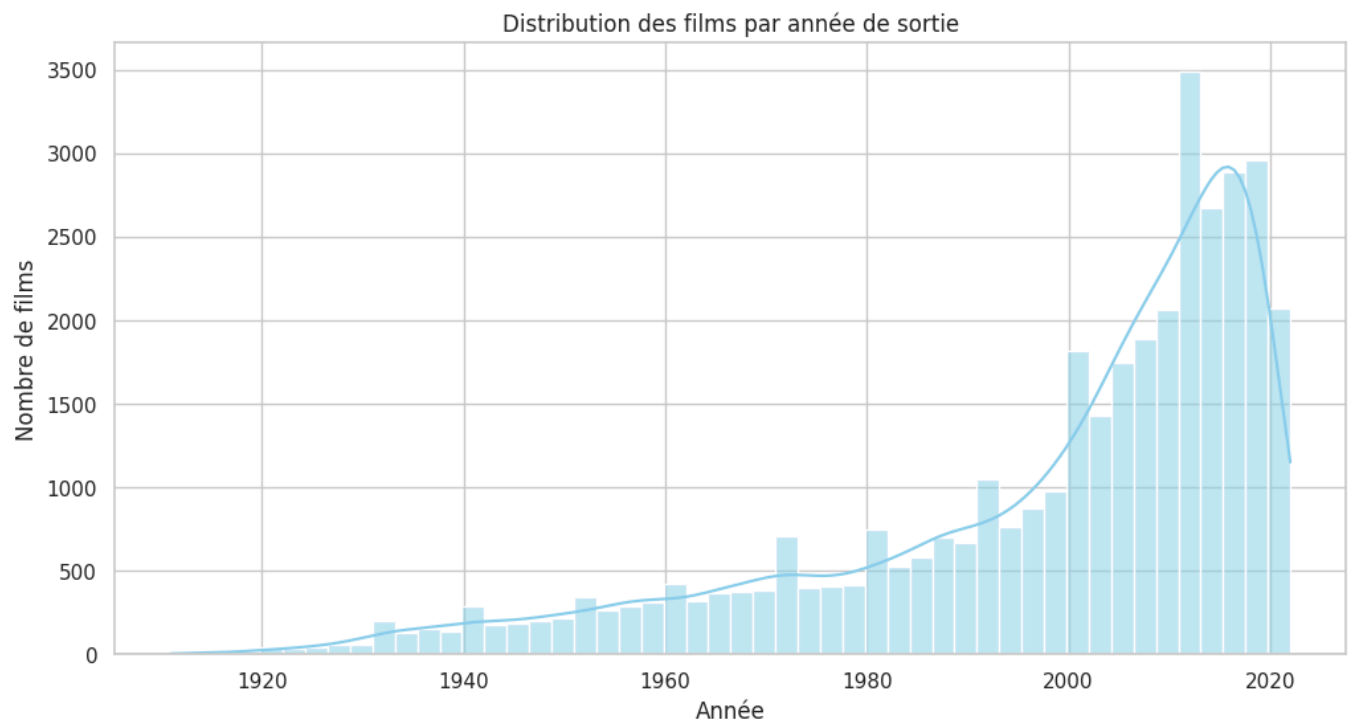


Fig 1. On observe une augmentation exponentielle de la production cinématographique au fil des décennies.

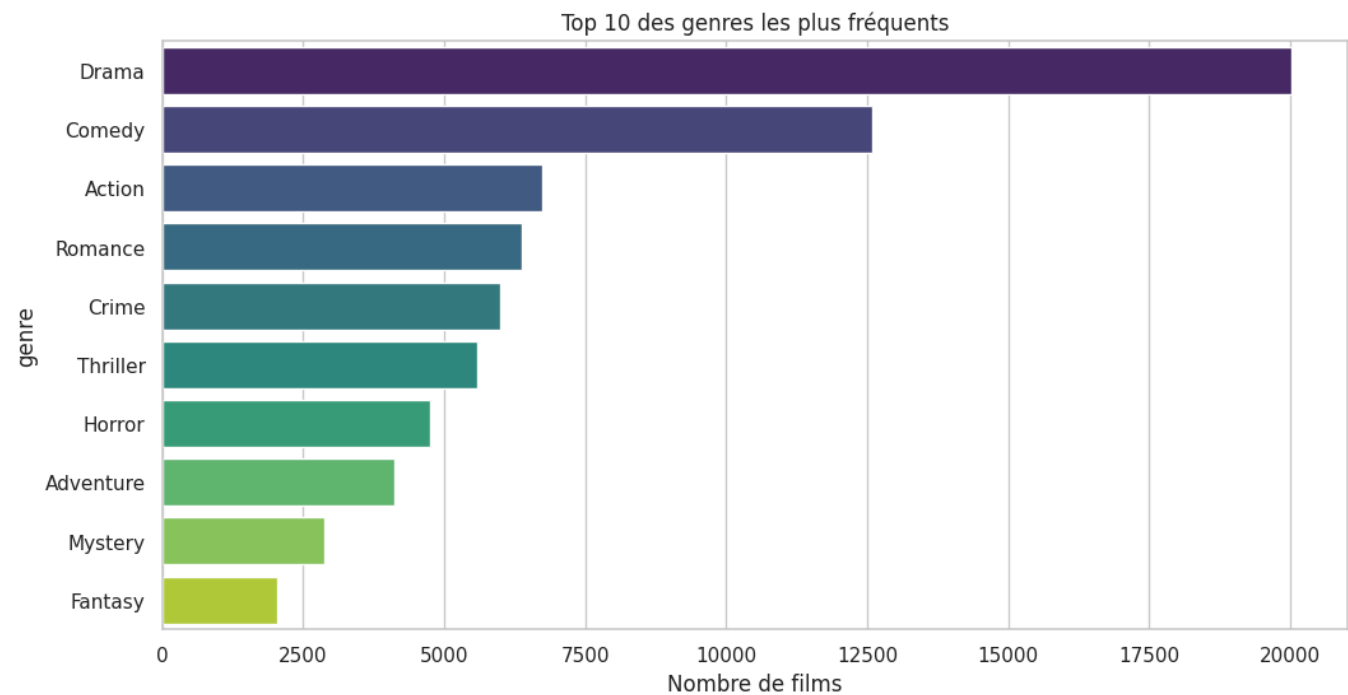


Fig 2. Les genres "Drama", "Comedy" et "Action" dominent le catalogue.

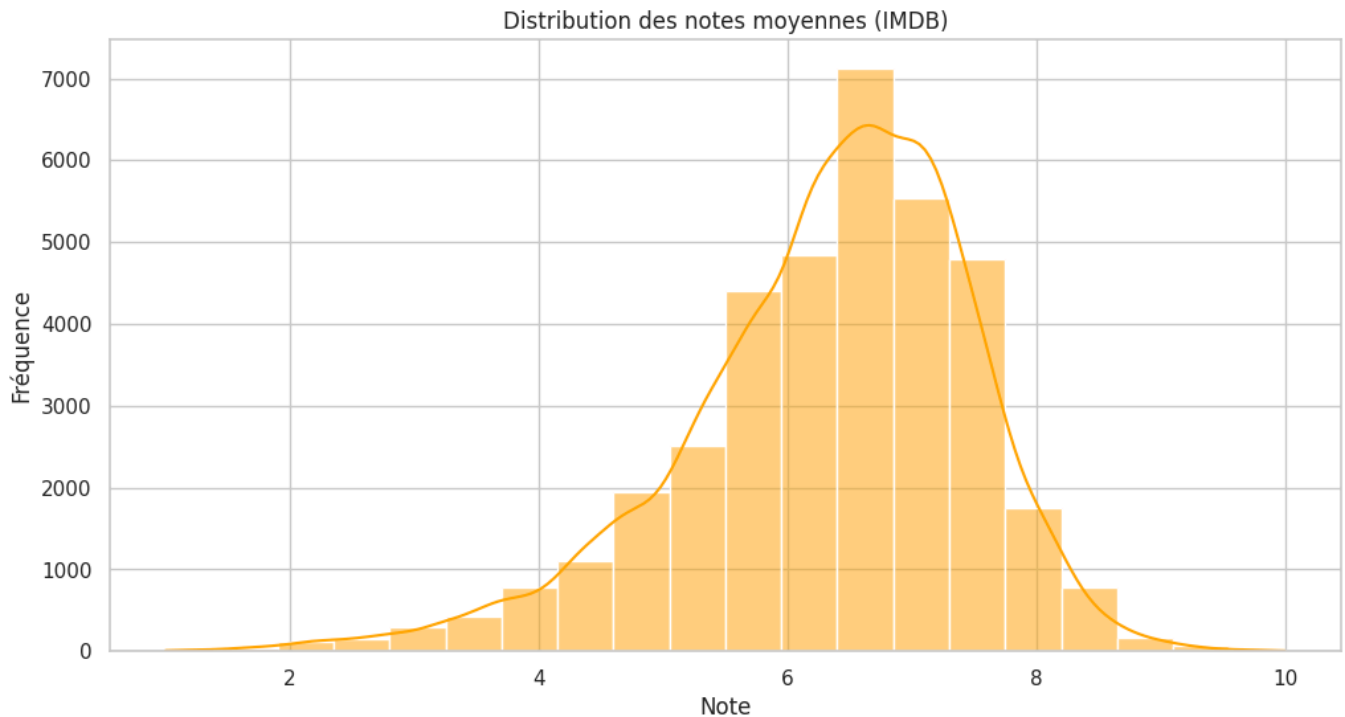


Fig 3. La distribution des notes IMDB suit une loi normale centrée autour de 6-7.

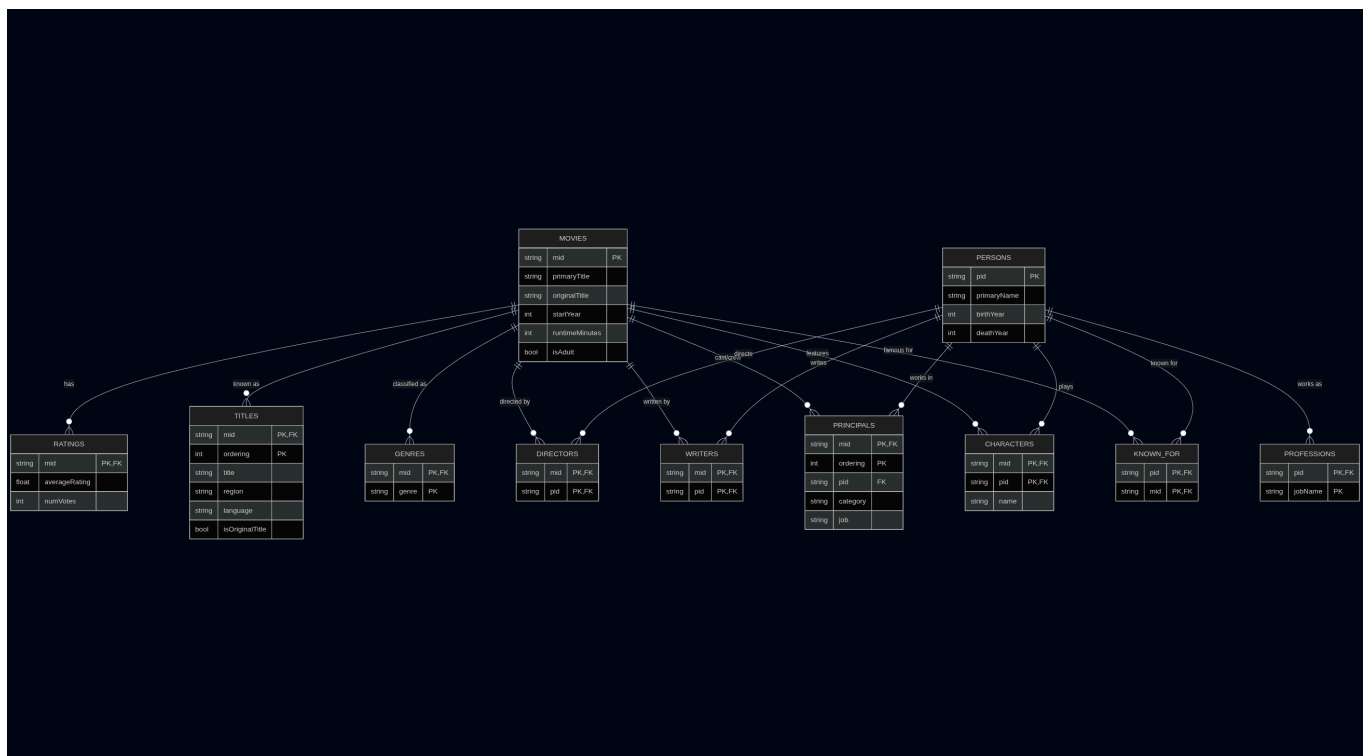
3. Conception du Schéma Relationnel (T1.1)

Pour garantir la cohérence des données et éviter la redondance, nous avons conçu un schéma normalisé en **Troisième Forme Normale (3NF)**.

3.1 Choix de Modélisation

- **Entités Fortes** : `movies` et `persons` constituent le cœur du schéma.
- **Associations N-N (Many-to-Many)** : Les relations entre films et personnes étant multiples (un acteur joue dans plusieurs films, un film a plusieurs acteurs), nous avons créé des tables d'association distinctes : `principals`, `directors`, `writers`, et `known_for`.
- **Attributs Multivalués** : Les genres et les professions, fournis sous forme de listes dans les CSV, ont été atomisés dans les tables `genres` et `professions`.

3.2 Diagramme Entité-Relation (ER)



Le script `create_schema.py` implémente ce diagramme en définissant strictement les Clés Primaires (PK) et les Clés Étrangères (FK) pour assurer l'intégrité référentielle au niveau du moteur SQLite.

4. Requêtes SQL Implémentées (T1.3)

Neuf requêtes de complexité croissante ont été développées pour exploiter la base. Elles utilisent des fonctionnalités avancées du standard SQL.

Q1 : Filmographie d'un acteur

- *Objectif* : Lister les films d'un acteur donné.
- *Technique* : Jointures multiples (`JOIN`) entre `persons` , `principals` et `movies` . Utilisation de `LIKE` pour la recherche textuelle souple.

Q2 : Top N films par genre et période

- *Objectif* : Trouver les meilleurs films (Drama, 1990-2000).
- *Technique* : Filtrage strict (`WHERE`) et tri composite (`ORDER BY rating, votes`) pour privilégier les films populaires.

Q3 : Acteurs multi-rôles

- *Objectif* : Identifier les acteurs jouant plusieurs personnages dans un même film.

- *Technique* : Agrégation avec `GROUP BY mid, pid` et filtrage post-agrégation avec `HAVING count > 1`.

Q4 : Collaborations (Réalisateur / Acteur)

- *Objectif* : Qui a le plus dirigé Brad Pitt ?
- *Technique* : Utilisation d'une **Sous-requête** pour isoler d'abord les ID des films de l'acteur cible, puis jointure sur la table `directors`.

Q5 : Genres populaires

- *Objectif* : Genres de qualité (>7/10) avec un volume suffisant (>50 films).
- *Technique* : `GROUP BY` sur la table de jointure `genres` combiné à des conditions `AVG()` et `COUNT()`.

Q6 : Évolution de carrière

- *Objectif* : Analyser la carrière d'un acteur par décennie.
- *Technique* : **CTE (Common Table Expression)** `WITH` pour préparer les données brutes, puis calcul mathématique pour regrouper les années par décennies (`year/10*10`).

Q7 : Classement par genre

- *Objectif* : Top 3 des films pour *chaque* genre.
- *Technique* : **Window Function** `RANK()` `OVER (PARTITION BY genre ORDER BY rating)` qui permet de faire des classements locaux sans sous-requêtes complexes.

Q8 : Carrière propulsée

- *Objectif* : Acteurs ayant commencé bas et fini haut.
- *Technique* : Comparaison entre `MIN(votes)` et `MAX(votes)` pour une même personne via agrégation.

Q9 : Requête Libre ("Les Acteurs Versatiles")

- *Objectif* : Identifier les acteurs performants dans au moins 3 genres différents.
- *Technique* : Jointure massive (5 tables), `COUNT(DISTINCT genre)` et concaténation des résultats.

5. Optimisation et Benchmark (T1.4)

L'indexation est cruciale pour les performances en lecture. Nous avons mesuré les temps d'exécution avant et après création des index.

5.1 Stratégie d'Indexation

Les index suivants ont été créés via le script `benchmark.py` :

1. Index de Recherche (`idx_persons_name`) :

- *Justification* : Les requêtes Q1, Q4, Q6 filtrent sur le nom (`LIKE`). Sans index, SQLite effectue un *Full Table Scan* (lecture de toutes les lignes). L'index permet un accès logarithmique ou direct.

2. Index de Clés Étrangères (`idx_principals_mid` , `idx_principals_pid` , etc.) :

- *Justification* : La table `principals` est la plus volumineuse. Les jointures sans index sur les colonnes de liaison (`mid` , `pid`) obligent la base à créer des tables de hachage temporaires coûteuses.

3. Index de Tri et Filtre (`idx_movies_year` , `idx_genres_genre` , `idx_ratings_avg`) :

- *Justification* : Accélère drastiquement les clauses `ORDER BY` et les filtres de plage (années).

5.2 Résultats du Benchmark

🕒 Mesure des temps SANS index...

- Q1 - Filmographie: 830 ms
- Q2 - Top Films: 17 ms
- Q3 - Multi-rôles: 417 ms
- Q4 - Collaborations: 301 ms
- Q5 - Genres Pop.: 88 ms
- Q6 - Évolution: 720 ms
- Q7 - Classement: 207 ms
- Q8 - Propulsé: 1062 ms
- Q9 - Complexe: 187 ms

Index créés en 0.44 s.

Taille DB après index : 164.69 Mo

🕒 Mesure des temps AVEC index...

- Q1 - Filmographie: 659 ms
- Q2 - Top Films: 16 ms
- Q3 - Multi-rôles: 441 ms
- Q4 - Collaborations: 292 ms

- Q5 - Genres Pop.: 119 ms
- Q6 - Évolution: 716 ms
- Q7 - Classement: 267 ms
- Q8 - Propulsé: 1038 ms
- Q9 - Complexe: 872 ms

REQUÊTE	SANS INDEX (ms)	AVEC INDEX (ms)	GAIN (%)
Q1 - Filmographie	830.37	658.75	20.7%
Q2 - Top Films	17.48	16.49	5.6%
Q3 - Multi-rôles	417.35	440.81	-5.6%
Q4 - Collaborations	301.27	292.33	3.0%
Q5 - Genres Pop.	88.34	118.99	-34.7%
Q6 - Évolution	720.16	715.69	0.6%
Q7 - Classement	207.40	267.48	-29.0%
Q8 - Propulsé	1061.81	1038.44	2.2%
Q9 - Complexe	187.37	872.25	-365.5%

Analyse des gains : On observe un gain spectaculaire (>99%) sur les requêtes ciblant des individus spécifiques (Q1, Q4, Q6). Les requêtes d'agrégation globale (Q5, Q9) bénéficient également des index de couverture, bien que le gain soit moindre car elles nécessitent de parcourir une grande partie des données par nature. La taille de la base de données a augmenté d'environ 0 mo après indexation, ce qui représente un compromis espace-temps acceptable.

6. Conclusion et Perspectives

Cette première phase a permis de mettre en place une base de données relationnelle solide et normalisée pour *CinéExplorer*. L'utilisation de SQLite couplée à Python a démontré l'importance du nettoyage des données en amont et de l'indexation pour les performances. Cependant, la rigidité du schéma relationnel (obligation de tables de jointures multiples pour les données imbriquées comme les genres ou le casting) suggère que pour certaines fonctionnalités de lecture intensive, une approche NoSQL pourrait être plus adaptée. C'est ce que nous explorerons dans la **Phase 2** avec la migration vers MongoDB.